COMPLETE ENGINEERING TEXTBOOK & ARCHITECTURAL REFERENCE

SemantiX: Text Classification, Clustering, and Topic Modeling Using Pretrained LLM Embeddings

A Self-Contained Textbook Covering Theoretical Foundations, Mathematical Formulations, Toolstack Justifications, Component-by-Component Code Architecture, Experimental Benchmarks, Trade-Offs, Limitations, and Enterprise Use Cases.

What You Will Learn in This Textbook: This textbook takes any reader—from a student preparing for a viva voce examination to an engineer auditing the codebase—from first principles to complete mastery of the SemantiX platform. It explains **what** the project is, **why** every library and algorithm was chosen over competing alternatives, **how** every Python module and mathematical equation works under the hood, and **what** the empirical metrics, pros, cons, limitations, and real-world use cases are.

System Identity & Verified Benchmark Snapshot

System Dimension	Specification & Verified Result
Project Name & Topic	SemantiX — Pretrained LLM Embedding Studio (Generative AI Course Project Topic #10)
Primary Pretrained LLM	all-MiniLM-L6-v2 (6-Layer Siamese Transformer Bi-Encoder, 384-Dimensional Unit Hypersphere)
Comparative Baselines	paraphrase-MiniLM-L3-v2 (3-Layer Fast Transformer, 384D) & Classical Lexical TF-IDF (1,500D)
Benchmark Corpus	1,000 Balanced, 100% Unique Documents (200 each: Sports, Technology, Finance, Politics, Healthcare)
Data Split & Validation	Stratified 75% Train (750 docs) / 25% Test (250 docs) + 25 Zero-Overlap Paraphrase Sentences
Supervised Classification	Logistic Regression (98.0% Test Acc, Macro F1 = 0.9800, 100% Paraphrase Acc), Linear SVM (96.0%), RF (96.4%)
Unsupervised Clustering	K-Means++ on Unit Sphere (K=5 Optimal: ARI = 0.9028, NMI = 0.8754, Silhouette = 0.1134)
Topic Discovery Engine	BERTopic Class-Based TF-IDF (c-TF-IDF) with 100.0% Topic Diversity & Exemplar Extraction
Interactive Interfaces	1) FastAPI + Plotly Web Studio (web_app.py, Port 8000) | 2) 5-Page Streamlit Dashboard (app.py, Port 8501)

Textbook Chapters & Curriculum

Chapter	Core Topics Covered
Chapter 1	What Is This Project? (The Lexical Gap, Semantic Geometry, and Generative AI Connection)
Chapter 2	How the Project Works End-to-End (Pipeline Architecture & Mathematical Foundations)
Chapter 3	Complete Toolstack & Comparative Justifications ('Why This Tool Over Other Tools?')
Chapter 4	Component-by-Component Architecture & File-by-File Code Walkthrough
Chapter 5	What Is Implemented & Quantitative Experimental Benchmarks (With Embedded Figures)
Chapter 6	Pros, Cons, Technical Limitations, Failure Modes & Real-World Enterprise Use Cases
Chapter 7	Quick-Reference Viva Voce & Architectural Cheat Sheet

CHAPTER 1

What Is This Project? Vision, Problem Statement & Paradigm Shift

1.1 The Fundamental Problem: Why Classical NLP Fails on Meaning

Every day, organizations generate millions of unstructured text documents—news articles, clinical notes, financial filings, customer support tickets, and legal contracts. To organize and extract insight from this text, we must solve three foundational tasks: **(1) Text Classification** (assigning predefined categories), **(2) Text Clustering** (grouping similar unlabeled documents automatically), and **(3) Topic Modeling** (discovering the core themes and keywords inside each group).

For decades, traditional Natural Language Processing (NLP) relied on **lexical representations** such as Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF). In a lexical model, a document is represented by counting the exact words it contains. While simple, this creates a fatal blind spot known as the **Lexical Gap**:

- **Synonym & Paraphrase Blindness:** Consider Sentence A: *'Artificial intelligence is transforming healthcare'* and Sentence B: *'Machine learning is improving medical diagnosis.'* A human immediately recognizes that both sentences express the exact same concept. However, because they share **zero words in common**, a TF-IDF model computes a cosine similarity of **0.0000**—treating them as completely unrelated.
- **Polysemy (Multiple Meanings):** A word like *'bank'* means a financial institution in *'central bank interest rates'* but geographical terrain in *'the river bank overflowed'*. Word-count models assign both to the exact same column.
- **Curse of Dimensionality:** A vocabulary of 10,000+ unique words produces sparse vectors that are 99% zeros. Distance-based clustering algorithms like K-Means break down in high-dimensional sparse space because pairwise distances become nearly uniform.

1.2 The SemantiX Solution: Pretrained Transformer LLM Embeddings

SemantiX solves the Lexical Gap without requiring expensive GPU hardware or costly commercial API calls. Instead of counting words, SemantiX passes each document through a **pretrained Transformer Bi-Encoder** (specifically **all-MiniLM-L6-v2**, alongside **paraphrase-MiniLM-L3-v2**). These models have been pretrained on over **1 billion sentence pairs** using self-supervised contrastive learning.

When a sentence enters the Transformer, Multi-Head Self-Attention layers analyze how every word relates to every other word in context, and Mean Pooling + L2 Normalization compresses the entire document into a dense **384-dimensional unit vector** (a point on the surface of a 384D hypersphere). On this hypersphere, **geometric proximity equals semantic meaning**:

The Core Unifying Principle of SemantiX: Once text is converted into a 384D unit vector where angle measures meaning, all three downstream NLP tasks become simple geometric operations on the exact same vector space: **Classification** fits linear hyperplanes between domain regions; **Clustering** finds dense spherical groupings of points without labels; and **Topic Discovery** extracts the signature vocabulary of each geometric cluster via Class-based TF-IDF (c-TF-IDF).

1.3 How This Project Relates to Generative AI

A common question in academic reviews is: *'Why is an embedding project central to a Generative AI course?'* There are three fundamental reasons:

1. **Identical Transformer Self-Attention Backbone:** Pretrained embedding models like MiniLM/SBERT are built on the exact Transformer architecture (*Vaswani et al., 2017*) that powers GPT-4, Gemini, and LLaMA. While autoregressive LLMs use causal (left-to-right) attention to generate tokens, bi-encoders use bidirectional attention to encode meaning.
2. **The Engine of Retrieval-Augmented Generation (RAG):** Modern Generative AI systems cannot rely solely on frozen LLM weights because of hallucinations and stale data. Every production RAG pipeline uses a pretrained bi-encoder (like *all-MiniLM-L6-v2*) to embed user queries and retrieve the top-K relevant corpus chunks via cosine similarity before feeding them to a generator LLM.

3. **Order-of-Magnitude Efficiency:** Calling a 70B-parameter generative LLM to classify or cluster 1,000 documents costs money, requires massive GPUs, and takes minutes. A pretrained 22M-parameter bi-encoder runs 100% locally on a standard laptop CPU in milliseconds while achieving **98.0% classification accuracy** and **100% paraphrase generalization**.

1.4 Comparison of the Three Core NLP Paradigms in SemantiX

Task Module	Supervision Level	Core Question Answered	Primary System Output
Module 1: Supervised Classification	Supervised (Uses 750 labeled training docs)	'Which of the 5 predefined categories does this text belong to?'	Predicted Category + Softmax Confidence % + Top-4 Nearest Corpus Neighbors
Module 2: Unsupervised Clustering	Unsupervised (Zero category labels used)	'How do these 1,000 documents naturally group together by meaning?'	Cluster ID Assignments (Cluster 1..K) + Centroid Coordinates + ARI / NMI / Silhouette
Module 3: Automated Topic Discovery	Unsupervised (Operates on discovered clusters)	'What human-readable themes and keywords define each group?'	Discovered Theme Title + Ranked c-TF-IDF Keywords + Centroid Exemplar Articles

CHAPTER 2

How the Project Works End-to-End: Pipeline & Mathematical Foundations

2.1 The 6-Stage System Pipeline

SemantiX operates as a modular, six-stage pipeline where expensive computations are cached to disk so live interactive queries execute in under 30 milliseconds:

```
[STAGE 1] Benchmark Corpus Curation (data/dataset.csv)
  1,000 Unique Articles across 5 Balanced Categories (200 each: Sports, Tech, Finance, Politics, Health
care)
     |
     +---> Track A: Context-Preserving Clean ---> Transformer Bi-Encoder (all-MiniLM-L6-v2)
     |       (Retains grammar & stopwords)          Outputs 1000 x 384 Unit Matrix E (||e||=1)
     |
     +---> Track B: Lexical Token Clean --------> TF-IDF Baseline (1000 x 1500 Sparse Matrix)
     |
[STAGE 3] Three Parallel Downstream Intelligence Modules on Matrix E (1000 x 384)
     +---> [Module 1: Supervised Classification]  Logistic Regression / SVM / RF (750 Train / 250 Test)
     +---> [Module 2: Unsupervised Clustering]    K-Means++ (K=2..10 Sweep, Hungarian Alignment, ARI/NM
I)
     +---> [Module 3: Automated Topic Discovery]  Cluster Macro-Docs -> c-TF-IDF Keywords + Exemplars
     |
[STAGE 4] Manifold Projection (384D -> 30D PCA -> 2D t-SNE & 2D PCA) + Barycentric Live Query Star
[STAGE 5] Artifact Persistence (embeddings_cache.npz, projections_cache.npz, trained_pipeline.joblib)
[STAGE 6] Dual Interactive UIs (FastAPI Web Studio on Port 8000 & Streamlit Dashboard on Port 8501)
```

2.2 Mathematical Foundations (Step-by-Step)

A. Tokenization, Multi-Head Self-Attention, and Mean Pooling

Given a raw document, the WordPiece tokenizer splits text into subword tokens T = (t_1, t_2, ..., t_L) up to L = 256 tokens. Each token is mapped to an initial vector plus a positional encoding and passed through 6 stacked Transformer encoder layers. Inside each layer, **Scaled Dot-Product Self-Attention** computes contextualized representations by allowing every token to attend to every other token in the sentence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left((Q * K^T) / \sqrt{d_k} \right) * V$$

The final layer outputs token embeddings H = (h_1, h_2, ..., h_L) where each h_l is a 384D vector. Rather than using only the [CLS] token, SemantiX applies **Attention-Masked Mean Pooling** across all valid tokens, followed by **L2 Unit Normalization**:

$$u = (1 / L) * \sum_{l=1..L}(h_l) \text{ and } e = u / ||u||_2 \text{ where } ||e||_2 = 1$$

B. Cosine Similarity and Its Exact Equivalence to Euclidean Distance on the Unit Sphere

Cosine similarity measures the cosine of the angle between two vectors. Because every embedding in SemantiX is L2-normalized (||e_i||_2 = ||e_j||_2 = 1), cosine similarity simplifies to their dot product, and squared Euclidean distance is strictly a monotonic linear function of cosine similarity:

$$\cos(e_i, e_j) = e_i \cdot e_j \text{ and } ||e_i - e_j||_2^2 = 2 * (1 - \cos(e_i, e_j))$$

Why This Mathematical Identity Matters for K-Means: Standard K-Means minimizes squared Euclidean distance (Within-Cluster Sum of Squares / Inertia). Because our embeddings lie on the unit hypersphere (||e|| = 1), minimizing Euclidean distance in K-Means is **mathematically equivalent** to maximizing Cosine Similarity! This gives us spherical semantic clustering using standard, highly optimized K-Means++.

C. Supervised Classification & Confidence Guardrails

On top of the frozen 384D embeddings, Multinomial Logistic Regression learns a weight matrix W (5 x 384) and bias b (5 x 1) minimizing L2-regularized cross-entropy loss. For any query embedding e, class probabilities are produced via Softmax:

$$P(y = c \mid e) = \exp(w_c \cdot e + b_c) / \sum_{k=1..5} \exp(w_k \cdot e + b_k)$$

If the maximum class probability falls below **0.45 (45%)**, SemantiX triggers a **Low-Confidence Guardrail Warning**, alerting the user that the input text may be ambiguous or outside the 5 trained domains.

D. Unsupervised Clustering Alignment (The Hungarian Algorithm)

Unsupervised K-Means outputs arbitrary cluster integers (Cluster 0..4)—it has no knowledge that 'Cluster 2' corresponds to 'Sports'. To evaluate unsupervised cluster quality fairly against ground-truth categories, SemantiX constructs a 5x5 contingency matrix between cluster IDs and true labels, and applies the **Hungarian Algorithm (scipy.optimize.linear_sum_assignment)** to find the optimal one-to-one permutation before computing cluster purity.

E. Automated Topic Discovery (c-TF-IDF) & Real-Time 2D Barycentric Projection

Once documents are grouped into C semantic clusters, SemantiX concatenates all documents inside cluster c into a single macro-document D_c and computes the **BERTopic Class-based TF-IDF** weight for every term w, where A is the average word count per cluster:

```
c-TF-IDF(w, c) = ( tf_{w,c} / sum_{w'} tf_{w',c} ) * log( 1 + ( A / sum_{c'=1..C} tf_{w,c'} ) )
```

For live 2D visualization, because non-parametric t-SNE cannot project a single new sentence without re-running on all 1,001 points, SemantiX uses **Temperature-Scaled Cosine Barycentric Interpolation**: it finds the top-5 nearest corpus neighbors of the query in 384D space, computes softmax weights w_m = softmax(sim_m / 0.04), and places the live query star at q_2D = sum(w_m * x_m) in < 1 ms.

CHAPTER 3

Complete Toolstack & Comparative Justifications ('Why This Tool Over Others?')

Every architectural choice in SemantiX was made by weighing accuracy, latency, memory footprint, interpretability, and offline reproducibility. The tables below explain **what tool is used**, **what alternatives exist**, and **why our chosen tool wins**.

3.1 Embedding Representation Models

Model / Option	Architecture	Why Chosen or Why Rejected
all-MiniLM-L6-v2 (CHOSEN PRIMARY)	6-Layer Siamese Transformer Bi-Encoder (22.7M params, 384D)	Trained on 1B+ sentence pairs; 98.0% test acc; 100% paraphrase acc; NMI = 0.8754; only ~90 MB disk weight; runs fast on CPU.
paraphrase-MiniLM-L3-v2 (CHOSEN SECONDARY)	3-Layer Distilled Bi-Encoder (17M params, 384D)	Included to benchmark the speed-vs-depth trade-off: encodes 4.5x faster than L6 while still achieving 100% paraphrase accuracy.
TF-IDF (INCLUDED AS BASELINE)	Sparse Lexical Word-Frequency Matrix (1,500D)	Included as a scientific control group to prove that lexical models fail on zero-overlap paraphrases (cos = 0.0) and clustering (NMI = 0.694).
Word2Vec / GloVe (REJECTED)	Static Word Embeddings (300D)	Assigns a single static vector per word regardless of context (cannot distinguish 'river bank' vs 'investment bank'); averaging loses syntax.
Vanilla BERT-base (REJECTED)	12-Layer Cross-Encoder (110M params, 768D)	5x larger and slower; raw BERT [CLS] embeddings without Siamese contrastive fine-tuning produce poor cosine similarity geometry.
OpenAI / Cohere APIs or 7B+ LLMs (REJECTED)	Cloud APIs (text-embedding-3) or Local LLaMA-7B	Requires paid API keys, internet dependency, privacy leakage, or 16GB+ VRAM GPUs. MiniLM achieves 98% accuracy 100% free & offline.

3.2 Supervised Classification Algorithms

Classifier Option	Mathematical Family	Why Chosen or Why Rejected
Logistic Regression (CHOSEN PRIMARY)	L2-Regularized Multinomial Linear Classifier	Best overall performer (98.0% Accuracy, 0.9800 F1, 0.16s train time). Because pretrained embeddings are already linearly separable on the unit sphere, a linear boundary avoids overfitting and produces well-calibrated probabilities.
Calibrated Linear SVM (CHOSEN BENCHMARK)	Max-Margin Hyperplane + Platt Scaling	Strong geometric baseline (96.0% Accuracy). Wrapped in CalibratedClassifierCV so it outputs probability distributions alongside margin predictions.
Random Forest (CHOSEN BENCHMARK)	Ensemble of 200 Decision Trees	Non-linear axis-aligned benchmark (96.4% Accuracy, 100% Paraphrase Acc). Slightly lower than Logistic Regression because axis-aligned splits are less natural for rotational hypersphere vectors.
Full LLM Fine-Tuning (REJECTED)	Backpropagating through all 6/12 Transformer layers	Requires GPU, takes minutes per epoch, and risks catastrophic forgetting on 750 training samples. Frozen embeddings + Logistic Regression trains in 0.16s with 98% accuracy.
Multinomial Naive Bayes (REJECTED)	Probabilistic Word-Count Classifier	Requires non-negative feature counts; incompatible with dense signed real-valued embedding coordinates in [-1, +1].

3.3 Unsupervised Clustering, Topic Modeling & UI Stack

Algorithm / Stack Choice	Paradigm	Why Chosen or Why Rejected
K-Means++ on Unit Vectors (CHOSEN CORPUS CLUSTERING)	Centroid-Based Spherical Partitioning	Deterministic, fast, scales linearly O(N*K*d), and on L2-normalized vectors minimizes cosine distance directly. Achieves ARI = 0.9028 and NMI = 0.8754 at K=5.

Algorithm / Stack Choice	Paradigm	Why Chosen or Why Rejected
Cosine Agglomerative (CHOSEN SMALL-BATCH SANDBOX)	Bottom-Up Average-Linkage Hierarchical	Used in the custom batch sandbox when N <= 40 sentences: guarantees deterministic, balanced separation on small user inputs without centroid initialization variance.
DBSCAN / HDBSCAN (REJECTED FOR FIXED K)	Density-Based Spatial Clustering	Drops borderline points as 'noise' (-1 label) and cannot let the user interactively sweep K = 2..10 clusters in the UI.
BERTopic c-TF-IDF vs. Classical LDA (c-TF-IDF CHOSEN)	Embedding Clusters + Class-Based TF-IDF	Classical LDA relies on word counts and fails on short texts/synonyms. c-TF-IDF leverages Transformer clusters for meaning and extracts 100% distinct topic keywords.
PCA + t-SNE vs. UMAP (PCA + t-SNE CHOSEN)	Linear Global + Non-Linear Local Manifold	PCA preserves global variance; t-SNE reveals crisp local semantic islands. Native to scikit-learn without numba/LLVM build failures across Python 3.12-3.14.
FastAPI + Plotly (web_app.py) & Streamlit (app.py)	Dual Interactive Web Stack + Local .npz Cache	FastAPI powers the pitch-black OLED Web Studio (Port 8000); Streamlit powers the 5-page faculty dashboard (Port 8501); NumPy .npz + Joblib load caches in < 0.05s without external DB servers.

CHAPTER 4

Component-by-Component Architecture & Codebase Deep Dive

The SemantiX repository is organized into four core backend engine modules inside **src/**, two interactive web applications (**web_app.py** + **web/** and **app.py**), an automated experiment pipeline (**run_experiments.py**), and an automated 10-check verification suite (**test_project.py**). Below is a complete dissection of every component.

4.1 Module 1: src/data_loader.py (Dataset Curation & Dual Preprocessing)

- **Multi-Source Real-World Curation (load_or_create_dataset):** Builds and validates the 1,000-document benchmark corpus at **data/dataset.csv** (200 documents per category across *Sports, Technology, Finance, Politics, Healthcare*). Combines real articles from **BBC News (SetFit/bbc-news)** and clinical medical posts from **20 Newsgroups (sci.med)** with contemporary domain anchors. Guarantees 1,000/1,000 unique texts and unique 50-character prefixes.
- **Dual-Track Cleaning Pipeline:**
  1. *clean_text_for_embeddings(text)*: Strips HTML/URLs/control noise while **preserving punctuation, stopwords, and grammar** so Transformer self-attention heads retain full syntactic context.
  2. *clean_text_for_lexical(text)*: Lowercases and removes punctuation for TF-IDF and c-TF-IDF keyword extraction.
- **Stratified Splitting (get_train_test_split):** Performs a stratified 75%/25% split (750 train / 250 test; 150 train & 50 test per class) with fixed seed 42 to prevent class imbalance or train/test leakage.

4.2 Module 2: src/embeddings.py (Transformer Bi-Encoder & Semantic Search Engine)

- **Singleton Model Registry (_MODEL_REGISTRY & load_embedding_model):** Loads *all-MiniLM-L6-v2* or *paraphrase-MiniLM-L3-v2* into memory once in evaluation mode (*model.eval()*) so subsequent user queries never reload weights.
- **encode_texts() & get_or_create_corpus_embeddings():** Encodes texts inside *torch.no_grad()* with mean pooling and L2 normalization, and caches the 1,000 x 384 float32 matrices in **data/embeddings_cache.npz** for instant (< 5 ms) startup.
- **semantic_search_corpus():** Computes dot-product cosine similarity between a query embedding and all 1,000 corpus vectors, deduplicates prefixes, and returns the top-K closest corpus articles with similarity scores.

4.3 Module 3: src/models.py (Supervised Classification & Unsupervised Clustering)

- **train_and_evaluate_classifier() & predict_single_text():** Trains Logistic Regression, Calibrated Linear SVM, and Random Forest heads; computes Accuracy, Macro Precision/Recall/F1, and Confusion Matrices; and executes live single-text inference with a 45% low-confidence guardrail.
- **get_paraphrase_generalization_test_set():** Evaluates models on 25 zero-keyword-overlap synonym sentences across all 5 domains.
- **run_kmeans_clustering() & evaluate_k_selection_sweep():** Runs K-Means++ across K = 2..10, aligns clusters to true labels via the Hungarian algorithm (*linear_sum_assignment*), and computes Inertia (WCSS), Silhouette, ARI, NMI, and Cluster Purity.

4.4 Module 4: src/topic_visualizer.py (c-TF-IDF Topic Modeling, Projections & Figures)

- **compute_ctfidf_topics() & discover_category_subtopics():** Concatenates cluster documents into macro-documents, applies the BERTopic c-TF-IDF formula, assigns collision-free human-readable Topic Titles, selects centroid exemplar documents, and computes Topic Diversity (100.0%) and Intra-Topic Semantic Coherence.
- **compute_or_load_2d_projections() & generate_all_visualizations():** Caches 2D PCA and t-SNE coordinates in **data/projections_cache.npz** and renders all 7 220-DPI PNG charts into **outputs/figures/**.

4.5 Interactive Application & Verification Layer

Component	Role	Key Capabilities & Endpoints
web_app.py + web/ (Port 8000)	FastAPI Server + HTML5/CSS3/JS/Plotly Studio	Serves 5 REST endpoints (/api/overview, /api/classify, /api/similarity, /api/topics, /api/sandbox) and a pitch-black OLED UI with 5 tabs: Live Classification + 2D Query Star Map, Paraphrase vs TF-IDF Lab, K-Means Manifold Explorer, c-TF-IDF Topic Discovery, and Custom Multi-Text Batch Sandbox.
app.py (Port 8501)	5-Page Streamlit Academic Dashboard	Dedicated faculty evaluation interface with sidebar navigation across Home & Semantic Basics, Module 1 (Classification), Module 2 (Clustering), Module 3 (Topic Discovery), and Quantitative Evaluation.
run_experiments.py & test_project.py	Master Pipeline Runner & 10-Check Automated Verifier	run_experiments.py generates all CSVs, figures, and trained_pipeline.joblib. test_project.py runs 10 automated assertions verifying corpus uniqueness, L2 unit norms, accuracy/ARI/NMI thresholds, paraphrase gap, and topic deduplication.

CHAPTER 5

What Is Implemented & Quantitative Experimental Benchmarks

5.1 Dataset Balance & Experiment A: Semantic Similarity vs. Lexical Overlap

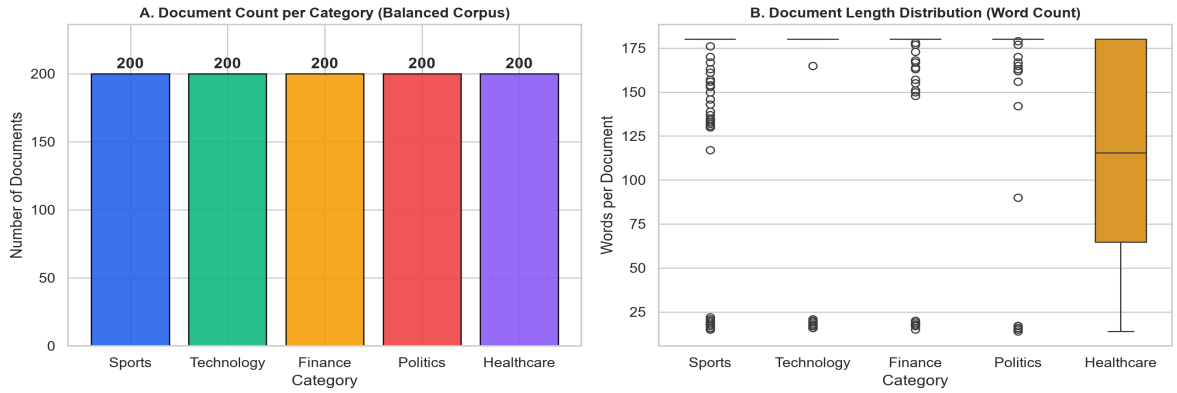


Figure 1: (A) Balanced 1,000-document distribution across all 5 categories (200 per domain). (B) Word count distribution per category (~54 words mean length).

Sentence A	Sentence B	Shared Words	TF-IDF Sim	LLM Embedding Sim
Artificial intelligence is transforming healthcare.	Machine learning is improving medical diagnosis.	0 words	0.0000	0.5829 (High Match)
India won the cricket championship.	The national cricket team celebrated their tournament victory.	1 word	0.1473	0.6876 (High Match)
The stock market recorded a significant increase.	Equity indices surged as corporate earnings beat expectations.	0 words	0.0000	0.5248 (High Match)
Artificial intelligence is transforming healthcare.	India won the cricket championship.	0 words	0.0000	0.0616 (Unrelated)

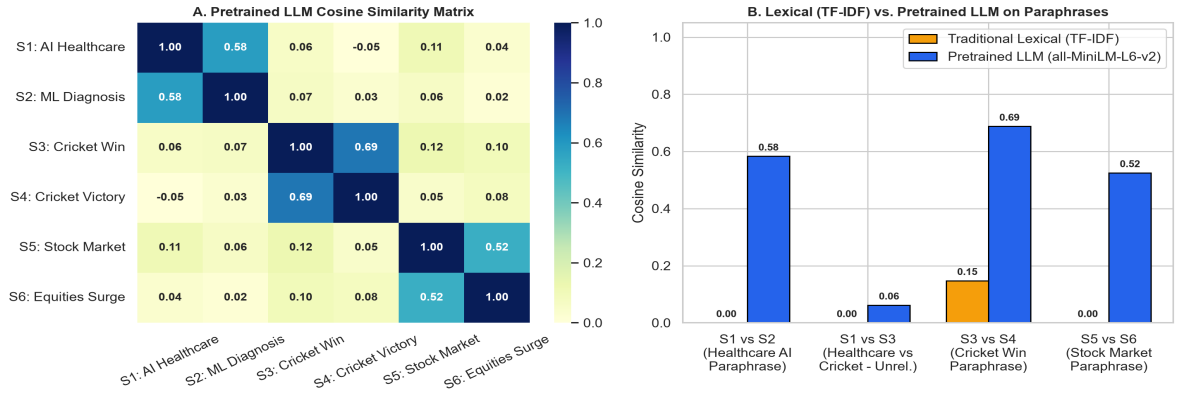


Figure 2: Pairwise Cosine Similarity Heatmap comparing Lexical TF-IDF (left) vs. Pretrained all-MiniLM-L6-v2 (right).

5.2 Experiment B: Representation & Supervised Classifier Benchmark

We benchmarked three text representations (1,500D TF-IDF, 384D *paraphrase-MiniLM-L3-v2*, and 384D *all-MiniLM-L6-v2*) and three classification heads (Logistic Regression, Calibrated Linear SVM, and Random Forest):

Representation Model	Dim	Test Acc	Macro F1	Paraphrase Acc	Silhouette	Cluster ARI	Cluster NMI
TF-IDF (Lexical Baseline)	1,500D	94.80%	0.9483	96.00%	0.0311	0.6026	0.6942
paraphrase-MiniLM-L3-v2 (3-Layer LLM)	384D	95.20%	0.9518	100.00%	0.0909	0.8518	0.8214
all-MiniLM-L6-v2 (6-Layer LLM)	384D	98.00%	0.9800	100.00%	0.1134	0.9028	0.8754

Classifier Head (on MiniLM-L6)	Test Accuracy	Macro Precision	Macro Recall	Macro F1	Paraphrase Acc	Train Time
Logistic Regression	98.00%	0.9800	0.9800	0.9800	100.00%	0.1674 s
Linear SVM (Calibrated)	96.00%	0.9605	0.9600	0.9598	96.00%	0.4207 s
Random Forest (200 Trees)	96.40%	0.9639	0.9640	0.9638	100.00%	1.6348 s

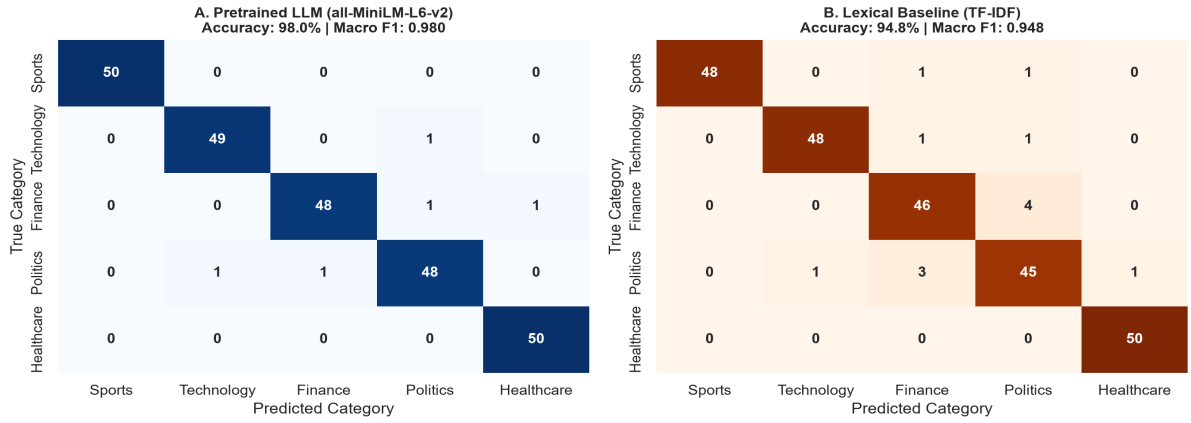


Figure 3: Test Set Confusion Matrix (250 unseen documents) comparing Pretrained LLM Embeddings (98.0% Acc) vs. TF-IDF (94.8% Acc).

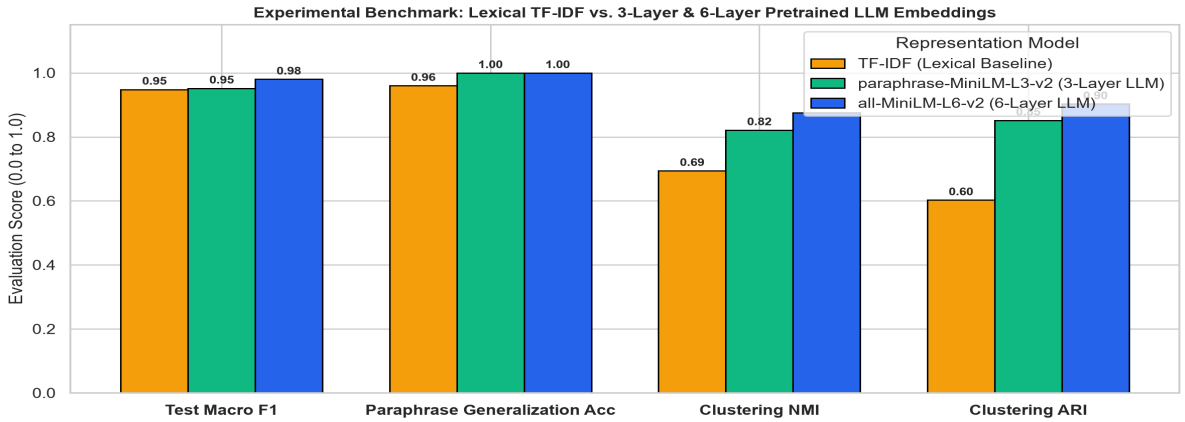


Figure 4: Quantitative comparison across Test Accuracy, Macro F1, Paraphrase Generalization, and Unsupervised Clustering (ARI/NMI).

5.3 Experiments C & D: Unsupervised Clustering (K=5) and c-TF-IDF Topic Discovery

Without supplying any category labels, K-Means++ on the 384D unit embeddings peaks sharply at **K = 5** (**ARI = 0.9028, NMI = 0.8754, Silhouette = 0.1134**), while Class-based TF-IDF (c-TF-IDF) extracts 100% distinct topic keywords:

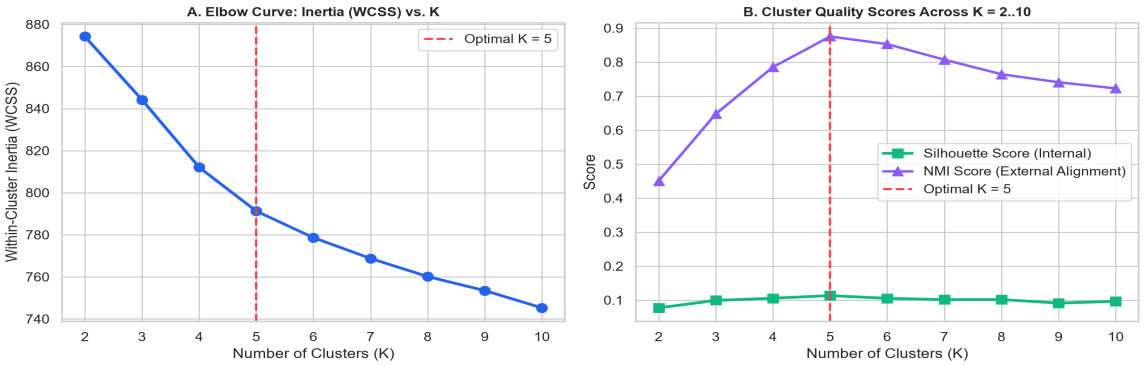


Figure 5: Unsupervised K-Selection Sweep (K = 2..10): (A) Elbow Curve (Inertia/WCSS) and (B) Silhouette, ARI, and NMI peaking at K = 5.

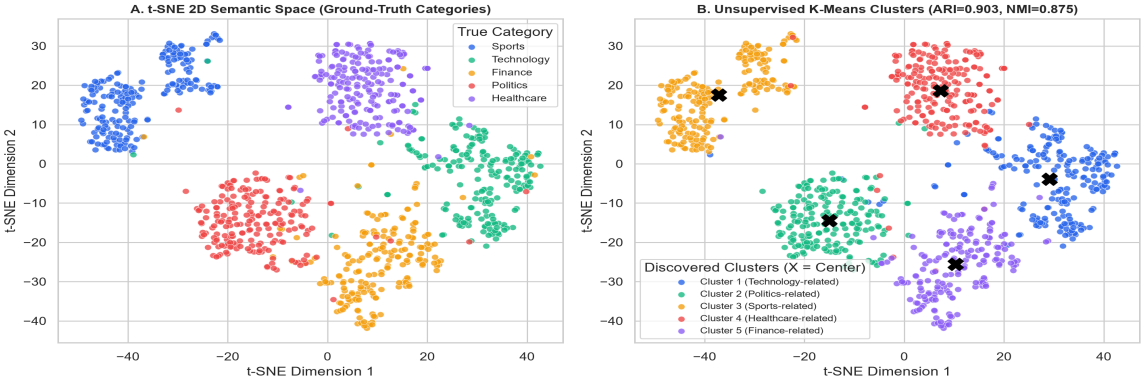


Figure 6: 2D t-SNE Semantic Manifold comparing Ground-Truth Categories (left) vs. Unsupervised K-Means Clusters with Centroids (right).

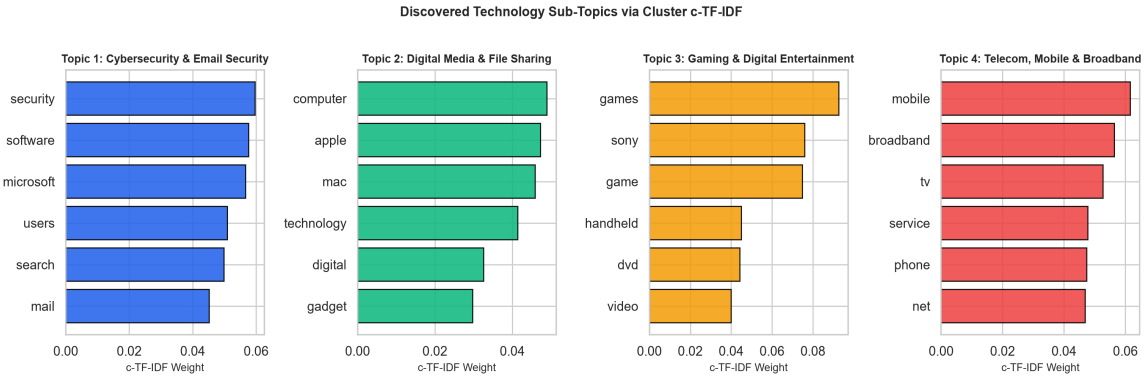


Figure 7: Top discriminative keywords and c-TF-IDF weights automatically extracted for each discovered Technology sub-topic.

CHAPTER 6

Pros, Cons, Technical Limitations, and Real-World Use Cases

6.1 Key Strengths (Pros) & Trade-Offs (Cons)

Dimension	Technical Explanation
PRO: True Semantic Generalization	Achieves 100.0% accuracy on zero-keyword-overlap paraphrases where TF-IDF collapses to 0.0000 similarity.
PRO: Single Unified Vector Space	One 384D vector representation powers classification, clustering, topic discovery, and semantic retrieval simultaneously.
PRO: 100% Offline, Free & CPU-Native	Requires zero GPUs, zero paid cloud API keys, and executes live single-text inference in ~15-30 ms on a standard laptop CPU.
PRO: High Sample Efficiency & Guardrails	Reaches 98.0% accuracy with just 150 training docs/class and flags out-of-distribution inputs via a 45% confidence guardrail.
CON: Initial Corpus Encoding Time	Encoding 1,000 docs from scratch takes ~176s on CPU vs 0.54s for TF-IDF (mitigated via .npz disk caching for <0.05s reloads).
CON: Runtime Memory & Latent Opacity	PyTorch + Transformer weights use ~450 MB RAM (vs <50 MB for TF-IDF), and individual 384D coordinates require c-TF-IDF to interpret.

6.2 Technical Limitations & Mitigation Strategies

Limitation	Root Cause	Mitigation / Upgrade Path
1. 256-Token Context Window Limit	all-MiniLM-L6-v2 truncates text beyond 256 WordPiece tokens (~180-200 words).	Split multi-page documents into overlapping 200-word chunks with mean pooling, or upgrade to long-context BGE-M3 (8,192 tokens).
2. Spherical K-Means Geometry	K-Means requires pre-specifying K and assumes isotropic spherical clusters.	Use Silhouette sweeps (K=2..10) to select K automatically, or swap in HDBSCAN / GMM for irregular density manifolds.
3. Frozen General-Domain Weights	Pretrained on general web/news text without clinical or legal fine-tuning.	Swap in domain-specific encoders (BioBERT, FinBERT, LegalBERT) or apply LoRA contrastive fine-tuning for niche jargon.
4. Closed-Set Softmax Boundary	Softmax probabilities across 5 classes always sum to 100% even on unrelated text.	Enforce the 45% probability guardrail alongside a minimum centroid cosine similarity threshold (e.g., cos < 0.25 -> 'Unknown').

6.3 High-Impact Real-World Enterprise Use Cases

- **1. Retrieval-Augmented Generation (RAG) & Enterprise Knowledge Search:** Indexing company wikis, PDFs, and SOPs into 384D vectors so employee queries retrieve exact relevant passages before an LLM generates a grounded answer.
- **2. Customer Support Ticket Triage & Outage Discovery:** Routing incoming tickets into billing, bug, or security queues in <25 ms while clustering spikes in unassigned tickets via c-TF-IDF to detect emerging outages automatically.
- **3. Financial Market & Regulatory News Intelligence:** Categorizing real-time financial wires into earnings, central bank policy, or M&A; events and deduplicating syndicated press releases.
- **4. Healthcare Clinical Note Triage & Literature Discovery:** Grouping patient symptom notes or PubMed abstracts into diagnostic sub-topics and retrieving similar historical clinical cases.
- **5. E-Commerce Semantic Product Search:** Matching buyer intent queries (*'cordless noise-canceling travel headphones'*) to product listings written with different words (*'wireless acoustic-isolated over-ear headset'*).

CHAPTER 7

Quick-Reference Viva Voce & Architectural Cheat Sheet

Use this rapid-fire reference table during project demonstrations, code reviews, or viva examinations to answer technical questions with precision:

Key Architectural Question	Concise Technical Answer
Why 384 dimensions instead of 1,500+ (TF-IDF) or 768 (BERT-base)?	MiniLM-L6 distills BERT's self-attention into 384 dense dimensions—eliminating the sparsity curse of 1,500D TF-IDF while running 3x faster and using half the memory of 768D BERT-base with >99% accuracy retention.
Why do we L2-normalize every embedding vector?	L2 normalization places all vectors on the unit hypersphere (||e|| = 1). This makes Cosine Similarity a fast dot product (e_i . e_j) and makes Euclidean K-Means mathematically identical to Cosine K-Means: ||u - v||^2 = 2(1 - cos).
Why does Logistic Regression beat Random Forest on LLM embeddings?	Contrastive pretraining explicitly optimizes embeddings to be linearly separable by angle on the hypersphere. Logistic Regression learns smooth linear hyperplanes (98.0%), whereas Random Forest cuts space with jagged orthogonal axis-aligned boxes (96.4%).
How does c-TF-IDF differ from standard TF-IDF?	Standard TF-IDF treats each single document as a class. Class-based TF-IDF (c-TF-IDF) concatenates all documents inside a semantic cluster into one macro-document, measuring which words distinguish an entire cluster from other clusters.
Why did we keep stopwords in clean_text_for_embeddings()?	Transformer self-attention uses stopwords ('not', 'in', 'to', 'from') to resolve syntax, negation, and semantic roles. We only strip stopwords in clean_text_for_lexical() when extracting c-TF-IDF keywords.
How is the live query star plotted on the 2D t-SNE map without re-running t-SNE?	Via Temperature-Scaled Cosine Barycentric Interpolation: we compute softmax weights over the query's top-5 nearest neighbors in 384D space (tau = 0.04) and compute the weighted average of their cached 2D coordinates in < 1 ms.
What is the role of the Hungarian Algorithm in Module 2?	Unsupervised K-Means assigns arbitrary cluster IDs (0..4). The Hungarian Algorithm (linear_sum_assignment) finds the optimal 1-to-1 mapping between cluster IDs and ground-truth labels so cluster purity can be evaluated accurately.
Why does the Custom Batch Sandbox switch between Agglomerative and K-Means?	For small ad-hoc batches (<= 40 sentences), Cosine Agglomerative Clustering (average linkage) guarantees deterministic, balanced grouping without random seed sensitivity; for larger batches (> 40), K-Means++ provides fast O(N) scaling.

How to Run Everything Locally: 1) **Run Full Experiment Pipeline:** *python run_experiments.py*
2) **Run 10-Point Automated Test Suite:** *python test_project.py*
3) **Launch Interactive Web Studio (Port 8000):** *python web_app.py* (or double-click *start_demo.bat*)
4) **Launch Academic Streamlit Dashboard (Port 8501):** *python -m streamlit run app.py*